

Ferhat Abbas Sétif 1 University
Faculty of Sciences
Department of Computer Science

Practical Work – Lab 4

Recurrent Neural Networks

Prof. Abdelouahab Moussaoui

4th Year Engineering
Option: Artificial Intelligence
Academic Year: 2025-2026

> *"Recurrent networks: teaching machines to remember one time step at a time."*
>
> — Pr. Abdelouahab Moussaoui

Preamble

This lab session is designed to complement Chapter 4 (Recurrent Neural Networks). The objective is to bridge the gap between the theory of sequential data processing and its practical implementation using TensorFlow/Keras. You will start with fundamental exercises to master the data preparation workflow for text, then explore various RNN architectures (Simple RNN, LSTM, GRU) and advanced techniques like Bidirectional RNNs and Encoder-Decoder models.

Lab Objectives

- Preprocess text data: tokenization, sequencing, and padding.
- Implement and train Simple RNN, LSTM, and GRU models on sentiment analysis tasks.
- Understand the impact of sequence length and vocabulary size.
- Master the use of Embedding layers.
- Experiment with Bidirectional RNNs and compare their performance.
- Build a character-level text generator.
- Tackle more complex tasks like sequence-to-sequence prediction and time series forecasting.
- Complete four mini-projects on diverse sequential data problems.

Tools and Frameworks

- Python 3.x, Google Colab or Jupyter Notebook.
- TensorFlow 2.x / Keras.
- NumPy, Matplotlib.
- (Optional) Gradio for deployment.

Submission Guidelines

- Submit a single Jupyter notebook (‘.ipynb‘) containing the code, outputs, and your analysis for the exercises.
- For the projects, submit a well-documented report (PDF) along with the source code.
- Deadline: To Be Announced (TBA).

Important Notes for Students

- The exercises are designed to progressively build your understanding. You are encouraged to complete as many as possible, but it is not mandatory to finish all of them.
- Regarding the mini-projects: **two projects are mandatory** (you may choose any two among the four). The remaining two projects can be completed as **bonus work**, which will contribute extra points to your final lab grade.

Contents

Preamble **2**

1 Part 1: Hands-On Deep Learning (Basic Exercises) **5**
1.1 Exercise 1: Text Preprocessing for RNNs 5
1.2 Exercise 2: Your First SimpleRNN for Sentiment Analysis 5
1.3 Exercise 3: SimpleRNN vs. LSTM vs. GRU 6
1.4 Exercise 4: The Effect of Embedding Dimensions 6
1.5 Exercise 5: Stacked RNNs for Increased Capacity 6
1.6 Exercise 6: Bidirectional RNNs for More Context 7
1.7 Exercise 7: Using Dropout with RNNs 7
1.8 Exercise 8: Using Pre-trained Embeddings (GloVe) 7
1.9 Exercise 9: Character-Level Text Generation 8
1.10 Exercise 10: Time Series Forecasting with RNNs 8

2 Part 2: Advanced Deep Learning **9**
2.1 Exercise 11: Building an Encoder-Decoder for Date Translation 9
2.2 Exercise 12: Implementing an Attention Mechanism from Scratch 9
2.3 Exercise 13: Text Classification with 1D Convolutions (Conv1D) 10
2.4 Exercise 14: Hyperparameter Tuning with Keras Tuner 10
2.5 Exercise 15: Sentiment Analysis on a New Dataset (Twitter US Airline) . . 10

3 Part 3: Mini-Projects **11**
3.1 Project 1: Next Word Prediction 11
3.2 Project 2: Multi-Class Text Classification (News Category) 11
3.3 Project 3: Stock Price Prediction using RNNs 12
3.4 Project 4: Fake News Detection 12

1 Part 1: Hands-On Deep Learning (Basic Exercises)

1.1 Exercise 1: Text Preprocessing for RNNs

- **Objective:** Master the essential steps of converting raw text into a format suitable for RNNs: tokenization, sequencing, and padding.
- **Dataset:** Use a few sample sentences of your own.
- **Tasks:**
  1. Create a list of 5 to 10 sentences (e.g., movie reviews, news headlines).
  2. Use `tf.keras.preprocessing.text.Tokenizer` to create a word index. Set a vocabulary size (e.g., `num_words=50`).
  3. Convert the sentences into sequences of integers using `texts_to_sequences`.
  4. Print the original sentences, the sequences, and the word index.
  5. Pad the sequences to a uniform length using `tf.keras.preprocessing.sequence.pad_seque` (e.g., `maxlen=10`). Experiment with `padding='pre'` and `padding='post'`, and `truncating='pre'` and `truncating='post'`.
  6. Observe and comment on the results of padding and truncation.

1.2 Exercise 2: Your First SimpleRNN for Sentiment Analysis

- **Objective:** Build, train, and evaluate a simple RNN on the IMDB dataset.
- **Dataset:** IMDB Movie Reviews (`tf.keras.datasets.imdb`). Dataset Link
- **Tasks:**
  1. Load the IMDB dataset with `num_words=10000`. Split into training and test sets.
  2. Pad the sequences to a fixed length (e.g., `maxlen=200`).
  3. Build a model with:
     - An `Embedding` layer (input_dim=10000, output_dim=32, input_length=200).
     - A `SimpleRNN` layer (units=32).
     - A `Dense` layer with 1 unit and 'sigmoid' activation.
  4. Compile with 'adam' optimizer and 'binary_crossentropy'.
  5. Train for 5 epochs and evaluate on the test set. Report test accuracy.
  6. Plot training and validation accuracy.

1.3 Exercise 3: SimpleRNN vs. LSTM vs. GRU

- **Objective:** Compare the performance of different RNN cells on a sentiment analysis task.
- **Dataset:** IMDB (as prepared in Exercise 2).
- **Tasks:**
  1. Build three identical models, replacing the `SimpleRNN` layer with:
     (a) `SimpleRNN(32)`
     (b) `LSTM(32)`
     (c) `GRU(32)`
  2. Train each model for 5 epochs (use the same validation split).
  3. Compare the final test accuracy and training time for each.
  4. Analyze the loss curves. Which model converges faster? Why?

1.4 Exercise 4: The Effect of Embedding Dimensions

- **Objective:** Understand how the size of the embedding vector impacts model performance.
- **Dataset:** IMDB.
- **Tasks:**
  1. Using an LSTM model as a baseline, vary the `output_dim` of the Embedding layer: [16, 32, 64, 128].
  2. Train each model and record the test accuracy.
  3. Plot embedding dimension vs. test accuracy.
  4. Discuss the trade-off: Does a larger embedding dimension always lead to better performance? Why might it lead to overfitting?

1.5 Exercise 5: Stacked RNNs for Increased Capacity

- **Objective:** Explore deep RNN architectures by stacking multiple recurrent layers.
- **Dataset:** IMDB.
- **Tasks:**
  1. Build a single-layer LSTM model (32 units) as a baseline.
  2. Build a stacked (deep) RNN model with two or three LSTM layers. Set `return_sequences=True` for all but the last LSTM layer.
  3. Compare the number of parameters, training time, and test accuracy.
  4. Does stacking help? Under what circumstances might a deeper network be beneficial?

1.6 Exercise 6: Bidirectional RNNs for More Context

- **Objective:** Implement and evaluate a Bidirectional LSTM.
- **Dataset:** IMDB.
- **Tasks:**
  1. Build an LSTM model (e.g., 32 units) as a baseline.
  2. Build a model that wraps the same LSTM layer with `Bidirectional`.
  3. Compare the number of parameters and test accuracy.
  4. Analyze why a bidirectional model might perform better for sentiment analysis.

1.7 Exercise 7: Using Dropout with RNNs

- **Objective:** Apply dropout to recurrent layers to combat overfitting.
- **Dataset:** IMDB (use a smaller subset to induce overfitting).
- **Tasks:**
  1. Train a baseline LSTM model without dropout.
  2. Train a second LSTM model with `dropout=0.2` and `recurrent_dropout=0.2` parameters set in the LSTM layer.
  3. Compare the training/validation accuracy curves. How does dropout affect overfitting?

1.8 Exercise 8: Using Pre-trained Embeddings (GloVe)

- **Objective:** Load pre-trained GloVe embeddings and use them in an RNN model.
- **Dataset:** A small subset of the IMDB dataset.
- **Tasks:**
  1. Download the GloVe 100-dimensional vectors (e.g., from the Stanford NLP group).
  2. Create an embedding matrix that maps the words in your IMDB vocabulary to their GloVe vectors.
  3. Build an LSTM model, initializing the `Embedding` layer with the GloVe weights. Set `trainable=False` initially.
  4. Train the model and evaluate its accuracy. Compare with a model that learns embeddings from scratch.
  5. (Optional) Fine-tune the pre-trained embeddings by setting `trainable=True` and training further.

1.9 Exercise 9: Character-Level Text Generation

- **Objective:** Build a character-level RNN to generate new text.
- **Dataset:** A text file (e.g., a short story, a collection of poems, or simply "The complete works of Shakespeare" in a small sample).
- **Tasks:**
  1. Load and preprocess the text: create a mapping from unique characters to indices.
  2. Create input sequences of a fixed length (e.g., 40 characters) and target outputs (the next character).
  3. Build an LSTM model (e.g., single LSTM layer with 128 units) followed by a Dense layer with softmax activation over the character vocabulary.
  4. Train the model.
  5. Write a function to generate new text given a seed string. Use a softmax temperature parameter to control the "creativity" of the generation.

1.10 Exercise 10: Time Series Forecasting with RNNs

- **Objective:** Use an RNN to forecast future values in a time series.
- **Dataset:** Air Passengers dataset (available online) or synthetic sine wave.
- **Tasks:**
  1. Load and normalize the time series data.
  2. Create a supervised learning dataset: use a window of past values (e.g., 12 months) to predict the next value.
  3. Build a SimpleRNN or LSTM model for regression (use 'mse' loss).
  4. Train the model and visualize the predictions on a test set.
  5. Compare the performance of an RNN with a simple linear autoregressive model.

2 Part 2: Advanced Deep Learning

2.1 Exercise 11: Building an Encoder-Decoder for Date Translation

- **Objective:** Implement a simple sequence-to-sequence (seq2seq) model to translate date formats (e.g., "January 21, 2024" to "2024-01-21").
- **Dataset:** Generate a synthetic dataset of dates in various text formats.
- **Tasks:**
  1. Create a synthetic dataset: generate 10,000 random dates and create pairs of (source format, target format).
  2. Preprocess both source and target sequences (character-level tokenization).
  3. Build an encoder-decoder model:
     - Encoder: Embedding + LSTM (returns state).
     - Decoder: Embedding + LSTM (uses encoder's state as initial state) + Dense.
  4. Train the model to predict the target sequence given the source sequence.
  5. Implement a simple inference function to decode a new date.
  6. Evaluate the accuracy of the translation.

2.2 Exercise 12: Implementing an Attention Mechanism from Scratch

- **Objective:** Add a Bahdanau attention layer to the encoder-decoder model from Exercise 11.
- **Dataset:** Same date translation dataset.
- **Tasks:**
  1. Modify the decoder from Exercise 11 to include an attention layer (like the `BahdanauAttention` class in the course notes).
  2. The decoder should now use the entire encoder output sequence, not just the final state.
  3. Train the model with attention and compare its performance and ability to handle longer sequences with the model from Exercise 11.
  4. Visualize the attention weights for a few sample translations.

2.3 Exercise 13: Text Classification with 1D Convolutions (Conv1D)

- **Objective:** Explore using 1D convolutional neural networks for text classification as an alternative to RNNs.
- **Dataset:** IMDB.
- **Tasks:**
  1. Build a model using `Conv1D` and `GlobalMaxPooling1D` layers after the embedding layer.
  2. Experiment with different kernel sizes and number of filters.
  3. Compare the performance (accuracy and training speed) of this model with the LSTM model from Exercise 3.

2.4 Exercise 14: Hyperparameter Tuning with Keras Tuner

- **Objective:** Use Keras Tuner to find optimal hyperparameters for an RNN model.
- **Dataset:** IMDB (use a smaller subset for speed).
- **Tasks:**
  1. Define a model-building function that takes a `hp` argument and allows tuning of:
     - Number of LSTM units (e.g., 32, 64, 128).
     - Dropout rate (e.g., 0.2, 0.3, 0.5).
     - Embedding dimension (e.g., 32, 64, 128).
  2. Use `kt.RandomSearch` or `kt.Hyperband` to search for the best hyperparameters.
  3. Report the best configuration and its test accuracy.

2.5 Exercise 15: Sentiment Analysis on a New Dataset (Twitter US Airline)

- **Objective:** Apply the skills learned to a new, more realistic text dataset.
- **Dataset:** Twitter US Airline Sentiment from Kaggle.
- **Tasks:**
  1. Load the dataset (CSV file). It contains tweets and their sentiment (positive, neutral, negative).
  2. Perform text cleaning (remove mentions, links, punctuation).
  3. Preprocess the text (tokenization, padding).
  4. Build an LSTM or GRU model to classify the sentiment.
  5. Evaluate the model and plot a confusion matrix.
  6. Analyze the misclassifications. What makes a tweet hard to classify?

3 Part 3: Mini-Projects

3.1 Project 1: Next Word Prediction

- **Objective:** Build a language model to predict the next word in a sentence.
- **Dataset:** Any large text corpus (e.g., a collection of news articles, Wikipedia dumps, or Project Gutenberg books). A good starting point is the "Text8" corpus or a collection of Sherlock Holmes stories.
- **Problem Statement:** Develop a model for a smart keyboard application that suggests the next word as the user types.
- **Tasks:**
  1. Load and preprocess a large corpus. Tokenize the text at the word level.
  2. Create input sequences of N words (e.g., 5 words) and the target word (the (N+1)-th word).
  3. Build and train a word-level language model using an Embedding layer followed by one or more LSTM layers.
  4. Experiment with the sequence length and the model's capacity.
  5. Write a function that takes a seed sentence and predicts the next 3-5 words.
  6. Evaluate the model's perplexity on a held-out test set.

3.2 Project 2: Multi-Class Text Classification (News Category)

- **Objective:** Classify news articles into predefined categories.
- **Dataset:** AG News Classification Dataset (available via TensorFlow Datasets or Kaggle). It contains 4 classes: World, Sports, Business, Sci/Tech.
- **Problem Statement:** Automatically tag incoming news articles for a news aggregator app.
- **Tasks:**
  1. Load and explore the dataset.
  2. Preprocess the text data.
  3. Build and compare at least two different models:
     - A Bi-LSTM model.
     - A Conv1D model.
     - (Bonus) A fine-tuned pre-trained transformer model (e.g., DistilBERT) using the Hugging Face `transformers` library.
  4. Evaluate using accuracy, precision, recall, and F1-score per class.
  5. Analyze the confusion matrix to see which categories are most often confused.

3.3 Project 3: Stock Price Prediction using RNNs

- **Objective:** Forecast the future closing price of a stock using historical data.
- **Dataset:** Historical stock data for a company (e.g., Apple, Google) from Yahoo Finance. You can use the `yfinance` library to download it.
- **Problem Statement:** Build a decision-support tool for traders to predict short-term price movements.
- **Tasks:**
  1. Download historical stock data (Open, High, Low, Close, Volume).
  2. Perform exploratory data analysis (plot the closing price over time).
  3. Create features (e.g., moving averages, price differences) or simply use the closing price.
  4. Normalize the data.
  5. Create sequences of past values (e.g., 60 days) to predict the next day's closing price.
  6. Build and train an LSTM model for regression.
  7. Visualize the predictions against the actual prices on a test set.
  8. **Critical Analysis:** Discuss the limitations and challenges of using deep learning for stock price prediction. Is it reliable? Why or why not?

3.4 Project 4: Fake News Detection

- **Objective:** Build a classifier to distinguish between real and fake news articles.
- **Dataset:** WELFake Dataset (available on Kaggle) or the LIAR dataset.
- **Problem Statement:** Develop a tool to help social media platforms flag potentially misleading content.
- **Tasks:**
  1. Load and explore the dataset. Handle missing values.
  2. Preprocess the text: lowercasing, removing punctuation/stop words, stemming/lemmatization.
  3. Analyze class balance.
  4. Build a strong baseline using a Bidirectional LSTM with GloVe embeddings.
  5. Experiment with regularization techniques (dropout, batch normalization) to combat overfitting.
  6. Evaluate the model using accuracy, precision, recall, and F1-score, paying close attention to the recall for the 'fake' class.
  7. (Bonus) Test your model on a few current news headlines to see how it performs.

"Recurrent networks are the key to unlocking the secrets of sequences, from the words we speak to the prices of tomorrow. Build them, train them, and let them remember."